

EPICODE — Institute of Technology

Attacco a un Database MySQL

Analisi di un Attacco SQL Injection tramite Wireshark

Analisi del file SQL_Lab.pcap: dal test di vulnerabilità all'enumerazione del database DVWA fino all'estrazione e al cracking degli hash delle password

Studente	Mauro Picca
Data	19 Giugno 2026
Corso	EPICODE — Cybersecurity & Ethical Hacking
Modulo	S11 L5 — Bonus 2: Attacco a un Database MySQL
Ambiente	CyberOps Workstation — Wireshark — file SQL_Lab.pcap

Attività svolta esclusivamente a fini didattici in ambiente di laboratorio controllato

Indice

1. Obiettivo	3
2. Apertura del File PCAP	3
3. Verifica della Vulnerabilità SQL Injection	5
4. Enumerazione del Database	6
5. Identificazione della Versione del Database	7
6. Enumerazione delle Tabelle	8
7. Estrazione degli Hash delle Password	9
8. Domande di Riflessione	10
9. Conclusioni	11

1. Obiettivo

Lo scopo del laboratorio è analizzare un file **PCAP** contenente un attacco SQL Injection contro un'applicazione web vulnerabile e comprenderne le diverse fasi tramite **Wireshark**.

2. Apertura del File PCAP

È stata avviata la macchina virtuale CyberOps Workstation e aperta l'applicazione **Wireshark**. Successivamente è stato caricato il file:

`SQL_Lab.pcap`

Il file contiene una cattura di rete della durata di circa 441 secondi relativa a un attacco SQL Injection contro un'applicazione **DVWA** (Damn Vulnerable Web Application).

No.	Time	Source	Destination	Protocol	Length	Info
28	441.804070	10.0.2.4	10.0.2.15	HTTP	685	GET /dvwa/vulnerabilities/sqli/?id=1%2...
25	383.277032	10.0.2.4	10.0.2.15	HTTP	680	GET /dvwa/vulnerabilities/sqli/?id=1%2...
22	313.710129	10.0.2.4	10.0.2.15	HTTP	659	GET /dvwa/vulnerabilities/sqli/?id=1%2...
19	277.727722	10.0.2.4	10.0.2.15	HTTP	630	GET /dvwa/vulnerabilities/sqli/?id=1%2...
16	220.490531	10.0.2.4	10.0.2.15	HTTP	577	GET /dvwa/vulnerabilities/sqli/?id=1%2...
13	174.254430	10.0.2.4	10.0.2.15	HTTP	536	GET /dvwa/vulnerabilities/sqli/?id=1%3...
11	0.068625	10.0.2.4	10.0.2.15	HTTP	429	GET /dvwa/dvwa/css/main.css HTTP/1.1
10	0.015485	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK] Seq=1019 Ack=3406 Win=...
8	0.014383	10.0.2.4	10.0.2.15	HTTP	496	GET /dvwa/index.php HTTP/1.1
7	0.005700	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK] Seq=589 Ack=365 Win=3...
4	0.000681	10.0.2.4	10.0.2.15	HTTP	654	POST /dvwa/login.php HTTP/1.1 (applic...
3	0.000349	10.0.2.4	10.0.2.15	TCP	66	35614 → 80 [ACK] Seq=1 Ack=1 Win=29312...
1	0.000000	10.0.2.4	10.0.2.15	TCP	74	35614 → 80 [SYN] Seq=0 Win=29200 Len=0...
30	441.807206	10.0.2.15	10.0.2.4	HTTP	2091	HTTP/1.1 200 OK (text/html)
29	441.804427	10.0.2.15	10.0.2.4	TCP	66	80 → 35668 [ACK] Seq=1 Ack=620 Win=236...
27	383.284289	10.0.2.15	10.0.2.4	HTTP	4068	HTTP/1.1 200 OK (text/html)
26	383.277811	10.0.2.15	10.0.2.4	TCP	66	80 → 35666 [ACK] Seq=1 Ack=615 Win=236...
24	313.712414	10.0.2.15	10.0.2.4	HTTP	1954	HTTP/1.1 200 OK (text/html)
23	313.710277	10.0.2.15	10.0.2.4	TCP	66	80 → 35644 [ACK] Seq=1 Ack=594 Win=236...
21	277.732200	10.0.2.15	10.0.2.4	HTTP	1955	HTTP/1.1 200 OK (text/html)
20	277.727871	10.0.2.15	10.0.2.4	TCP	66	80 → 35642 [ACK] Seq=1 Ack=565 Win=236...
18	220.493085	10.0.2.15	10.0.2.4	HTTP	1918	HTTP/1.1 200 OK (text/html)
17	220.490637	10.0.2.15	10.0.2.4	TCP	66	80 → 35640 [ACK] Seq=1 Ack=512 Win=235...
15	174.257989	10.0.2.15	10.0.2.4	HTTP	1861	HTTP/1.1 200 OK (text/html)
14	174.254581	10.0.2.15	10.0.2.4	TCP	66	80 → 35638 [ACK] Seq=1 Ack=471 Win=235...
12	0.070400	10.0.2.15	10.0.2.4	HTTP	1511	HTTP/1.1 200 OK (text/css)
9	0.015485	10.0.2.15	10.0.2.4	HTTP	3107	HTTP/1.1 200 OK (text/html)
6	0.005700	10.0.2.15	10.0.2.4	HTTP	430	HTTP/1.1 302 Found
5	0.002149	10.0.2.15	10.0.2.4	TCP	66	80 → 35614 [ACK] Seq=1 Ack=589 Win=302...
2	0.000315	10.0.2.15	10.0.2.4	TCP	74	80 → 35614 [SYN, ACK] Seq=0 Ack=1 Win=...

Figura 1 – Apertura del file SQL_Lab.pcap in Wireshark.

Quali sono i due indirizzi IP coinvolti nell'attacco?

Ruolo	Indirizzo IP
Attaccante	10.0.2.4
Server / Vittima	10.0.2.15

3. Verifica della Vulnerabilità SQL Injection

È stata aperta la riga 13 della cattura utilizzando la funzione **Follow HTTP Stream**. L'aggressore invia la seguente richiesta:

1=1

La risposta del server restituisce un record presente nel database.

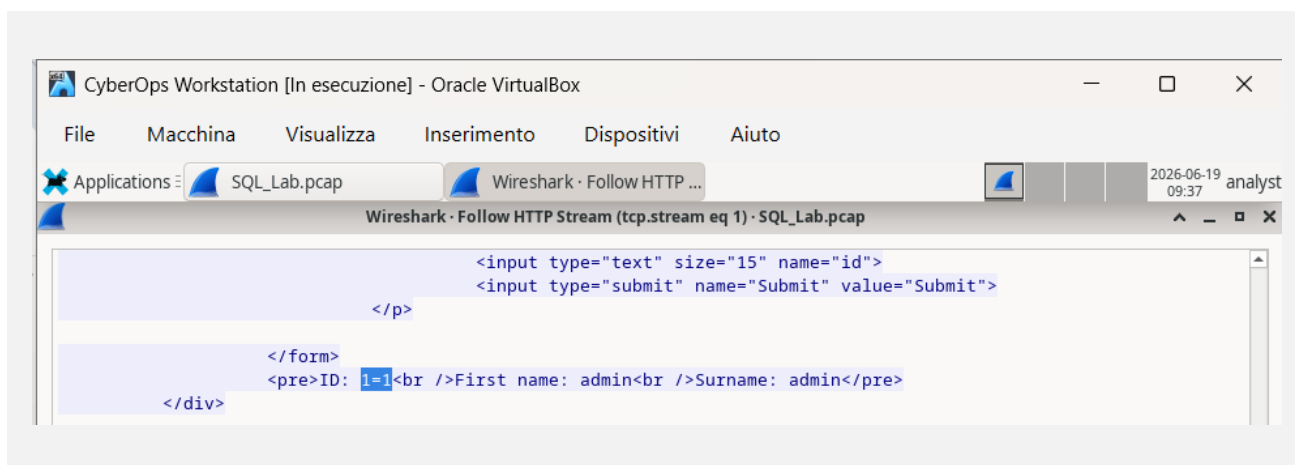


Figura 2 – Follow HTTP Stream riga 13: test di vulnerabilità con la stringa 1=1.

La stringa **1=1** rappresenta una condizione sempre vera. L'applicazione, invece di restituire un errore o bloccare la richiesta, restituisce il record dell'utente amministratore.

Cosa dimostra questa risposta?

L'applicazione risulta vulnerabile alla **SQL Injection** poiché accetta ed esegue l'input fornito dall'utente all'interno della query SQL. L'aggressore verifica che il database risponde ai comandi inseriti.

4. Enumerazione del Database

È stata analizzata la riga 19 tramite **Follow HTTP Stream**. L'aggressore utilizza la query:

```
1' or 1=1 union select database(), user()#
```

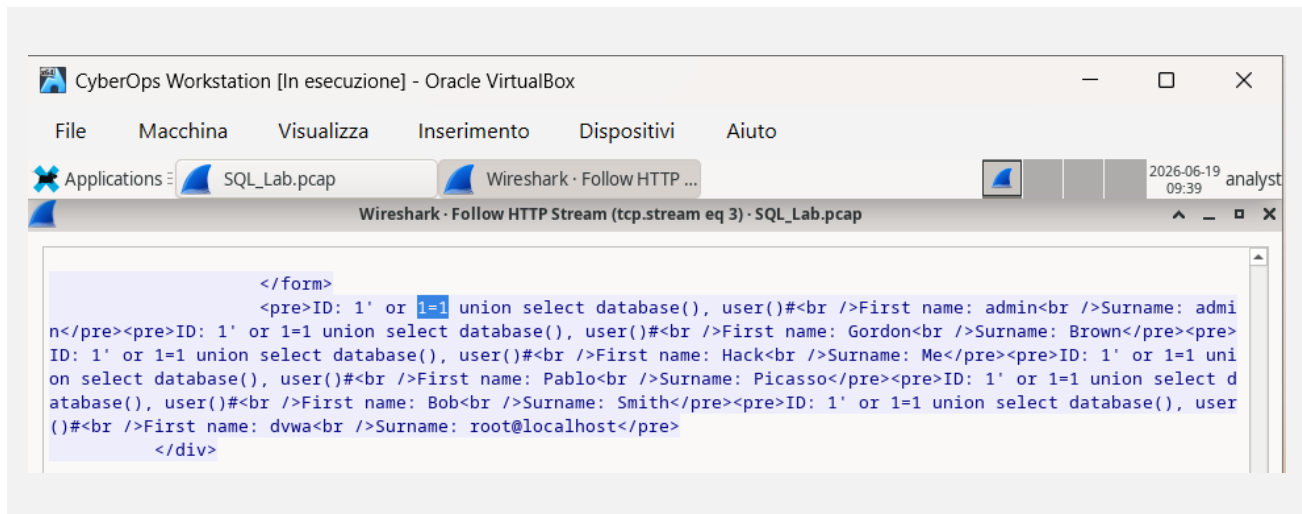


Figura 3 – Follow HTTP Stream riga 19: estrazione di nome database e utente.

La query sfrutta **UNION SELECT** per estrarre informazioni dal database.

Quali informazioni vengono ottenute?

Informazione	Valore
Nome del database	dvwa
Utente del database	root@localhost

L'aggressore ottiene inoltre diversi record presenti nel database.

5. Identificazione della Versione del Database

È stata aperta la riga 22 della cattura. L'aggressore utilizza la query:

```
1' or 1=1 union select null, version()#
```

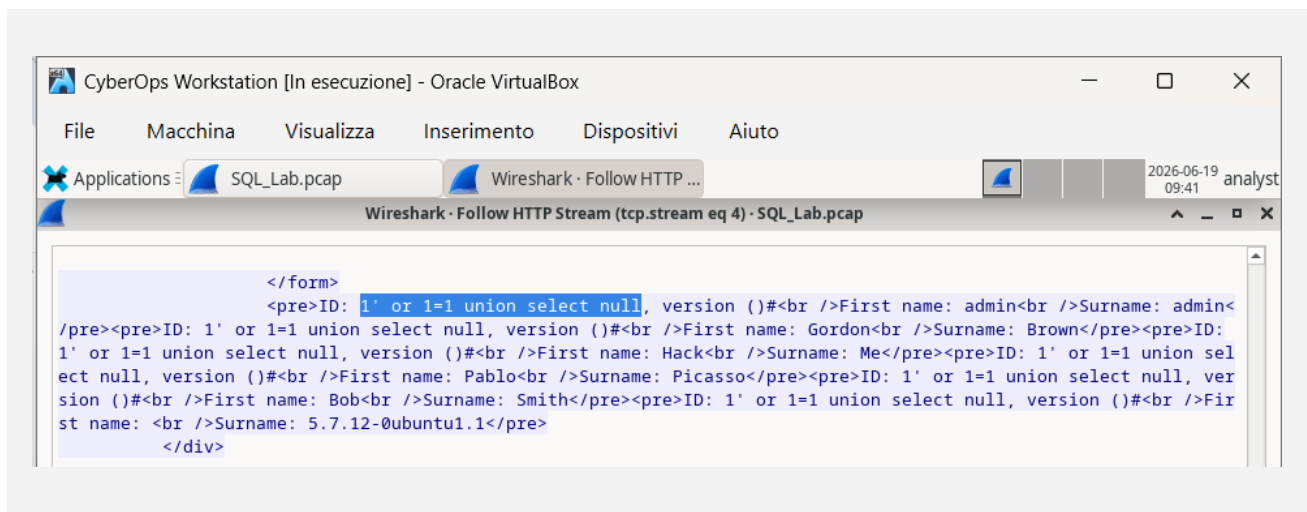


Figura 4 – Follow HTTP Stream riga 22: identificazione della versione MySQL.

Qual è la versione del database?

```
5.7.12-0ubuntu1.1
```

Conoscere la versione esatta del **DBMS** consente a un attaccante di cercare vulnerabilità specifiche e pianificare ulteriori attacchi.

6. Enumerazione delle Tabelle

È stata analizzata la riga 25 tramite **Follow HTTP Stream**. L'aggressore esegue la query:

```
1' or 1=1 union select null, table_name from
information_schema.tables#
```

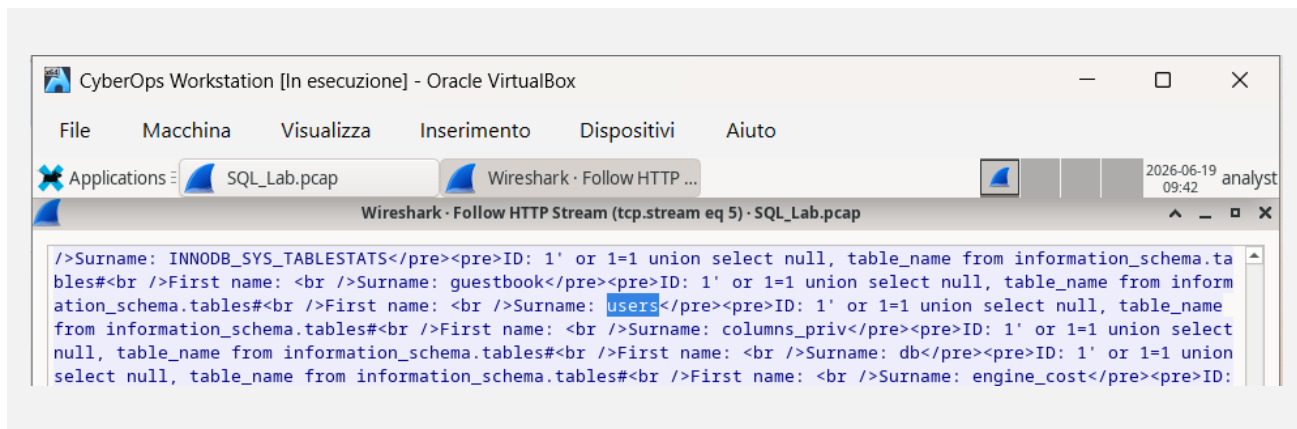


Figura 5 – Follow HTTP Stream riga 25: enumerazione delle tabelle tramite INFORMATION_SCHEMA.

La query interroga **INFORMATION_SCHEMA** e restituisce l'elenco delle tabelle presenti nel database. Tra i risultati compare la tabella:

users

Cosa farebbe il comando modificato 1' OR 1=1 UNION SELECT null, column_name FROM INFORMATION_SCHEMA.columns WHERE table_name='users'?

La query restituirebbe l'elenco delle colonne della tabella users. L'aggressore potrebbe identificare informazioni sensibili come:

1. username
2. password
3. user_id
4. first_name
5. last_name

ottenendo dettagli utili per compromettere gli account presenti nel database.

7. Estrazione degli Hash delle Password

È stata analizzata la riga 28 tramite **Follow HTTP Stream**. L'aggressore utilizza la query:

```
1' or 1=1 union select user,password from users#
```

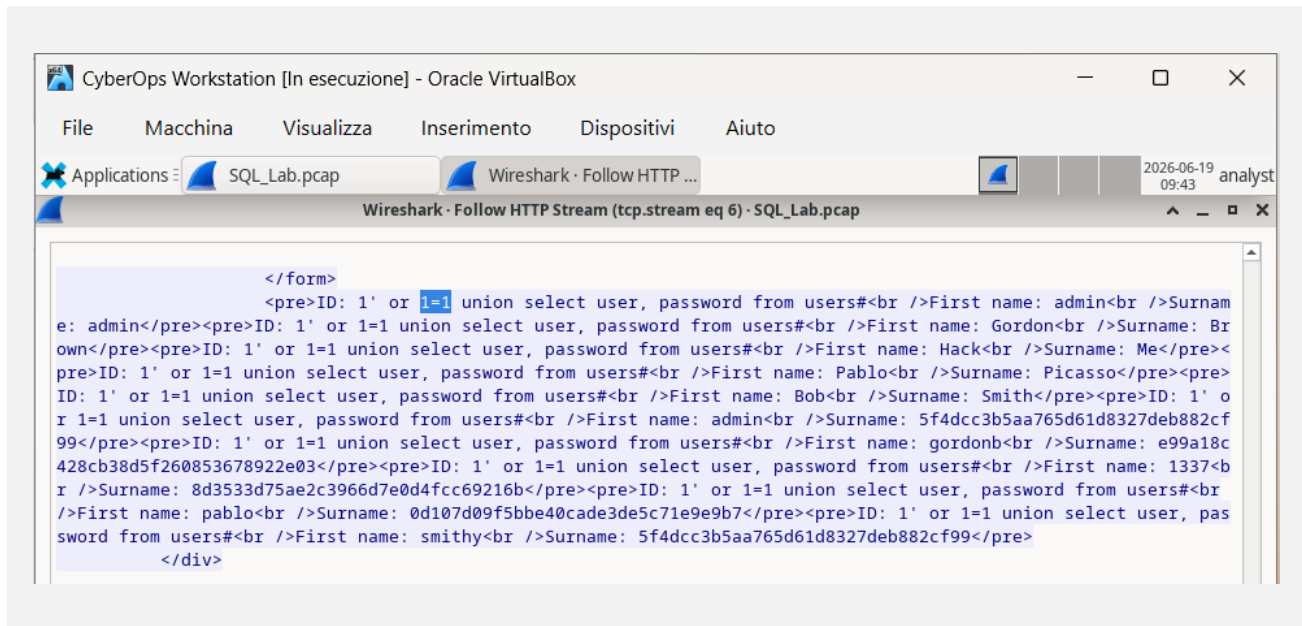


Figura 6 – Follow HTTP Stream riga 28: estrazione di utenti e hash delle password.

La query estrae direttamente gli utenti e gli hash delle password presenti nel database.

Quale utente possiede l'hash 8d3533d75ae2c3966d7e0d4fcc69216b?

Bob

Qual è la password in chiaro?

Utilizzando **CrackStation** per effettuare il cracking dell'hash MD5, è stata individuata la password:

charley

⚠ L'estrazione e il cracking degli hash sono spesso la fase finale di un attacco SQL Injection: se le password sono in chiaro o hashate senza salt con algoritmi deboli come MD5, vengono recuperate in pochi secondi tramite rainbow table o servizi online come CrackStation.

8. Domande di Riflessione

Qual è il rischio che le piattaforme utilizzino il linguaggio SQL?

I database SQL sono utilizzati dalla maggior parte delle applicazioni web moderne. Se gli input dell'utente non vengono correttamente validati e filtrati, un aggressore può eseguire attacchi SQL Injection. Le conseguenze possono includere:

1. lettura di dati riservati
2. modifica o cancellazione di informazioni
3. furto di credenziali
4. compromissione completa del database
5. escalation verso altri sistemi collegati

Quali metodi possono prevenire gli attacchi SQL Injection?

Tra le principali contromisure:

1. utilizzo di query parametrizzate (Prepared Statements)
2. validazione e sanitizzazione degli input utente
3. implementazione di Web Application Firewall (WAF)
4. principio del minimo privilegio per gli account database
5. monitoraggio e analisi dei log SQL
6. disabilitazione delle funzionalità non necessarie del DBMS

9. Conclusioni

L'analisi del file **PCAP** ha permesso di osservare passo dopo passo un attacco **SQL Injection** contro **DVWA**. È stato possibile verificare come un aggressore possa inizialmente testare una vulnerabilità, ottenere informazioni sul database, identificare la versione del DBMS, enumerare tabelle e colonne e infine estrarre hash delle password da utilizzare in successive attività di compromissione.

Fase	Query / Azione	Risultato Ottenuto
Test vulnerabilità	1=1	Bypass login — record restituito
Enumerazione DB	union select database(), user()	dvwa — root@localhost
Versione DBMS	union select null, version()	5.7.12-0ubuntu1.1
Enumerazione tabelle	union select null, table_name from information_schema.tables	Tabella users individuata
Estrazione hash	union select user, password from users	Bob — 8d3533d75ae2c3966d7e0d4fcc69216b → charley

Il laboratorio evidenzia l'importanza della validazione degli input e dell'utilizzo di tecniche di sviluppo sicuro per prevenire vulnerabilità SQL Injection.

Lesson Learned

- Un'unica condizione sempre vera (1=1) è sufficiente per individuare un'applicazione vulnerabile: questo è il primo test che un attaccante esegue, ed è anche il primo controllo che un penetration test deve verificare sia bloccato.
- UNION SELECT trasforma una singola query vulnerabile in un canale completo per esfiltrare l'intero schema del database: da database() e user() fino a INFORMATION_SCHEMA, un attaccante può mappare l'intera struttura senza credenziali aggiuntive.
- Le query parametrizzate (Prepared Statements) sono la difesa più efficace contro SQL Injection: separano il codice SQL dai dati dell'utente a livello di driver del database, rendendo impossibile l'iniezione indipendentemente da cosa l'utente invii come input.

☒ Tutti gli obiettivi del laboratorio, incluso il bonus, sono stati raggiunti. L'intera catena dell'attacco SQL Injection è stata ricostruita dal file PCAP, dal test iniziale fino al cracking della password finale.